

PRODUCT REQUIREMENT DOCUMENT (PRD)

EA BACKTEST ENGINE V3

Python MT5-Compatible Strategy Tester, Visual Simulator & Trading Behavior Template Engine

Version: 3.0

Status: Proposed

Date: 2 September 2026

Primary Language: Python

Target Platform: Web Application + Python Backend

Primary Market: Forex / Gold / CFD, dengan fokus awal XAUUSD

Reference Platform: MetaTrader 5 Strategy Tester

1. EXECUTIVE SUMMARY

EA Backtest Engine V3 adalah platform backtesting berbasis Python yang dirancang untuk mensimulasikan mekanisme kerja Expert Advisor (EA) seperti MetaTrader 5 Strategy Tester.

Sistem tidak hanya menghitung profit dari historical data, tetapi menyediakan:

1. EA Parser

2. Membaca source code `.mq5`

3. Mengekstrak input, variable, function, indicator, trading logic, risk management, order management, dan dependency.

4. Mengubah struktur EA menjadi Common Strategy Intermediate Representation (Strategy IR).

5. Indicator & Trading Library

6. Menjadi library bersama untuk EA Parser, EA Builder, Backtest Engine, dan Simulator.

7. Menyediakan implementasi indikator yang kompatibel dengan konsep MQL5.

8. Menyediakan signal, market, order, position, account, dan trading functions.

9. Python Backtest Engine

10. Menjalankan strategy berdasarkan Strategy IR.
 11. Mensimulasikan market, order, position, SL/TP, spread, commission, swap, slippage, margin, dan equity.
 12. **Visual Backtest Simulator**
 13. Meniru Visual Mode MT5.
 14. Market bergerak candle-by-candle atau tick-by-tick.
 15. Menampilkan chart, entry, exit, SL, TP, trailing, equity, balance, dan trade history.
 16. **MT5 Result Comparator**
 17. Mengimpor hasil Strategy Tester MT5 dalam CSV.
 18. Membandingkan hasil MT5 dengan Python secara trade-by-trade dan equity-by-equity.
 19. Menghasilkan parity score dan discrepancy report.
 20. **Trading Behavior Analyzer**
 21. Menganalisis historical trading/market behavior.
 22. AI digunakan hanya pada tahap interpretasi behavior.
 23. Statistik dasar dihitung deterministic oleh Python.
 24. Menghasilkan reusable Trading Behavior Template.
 25. **Trading Behavior Template Engine**
 26. Menyimpan pola historis market.
 27. Menghubungkan behavior dengan event, session, volatility, trend, price action, dan kondisi market.
 28. Template dapat digunakan sebagai referensi untuk backtest dan analisis trading berikutnya.
-

2. PRODUCT OBJECTIVE

Tujuan utama adalah membuat engine yang:

"Mampu menjalankan EA secara Python-based dengan perilaku eksekusi yang sedekat mungkin dengan MT5, memvalidasi hasilnya terhadap MT5 Strategy Tester, sekaligus mempelajari kebiasaan market dari historical data untuk membangun Trading Behavior Template."

Sistem harus memisahkan dua hasil utama:

OUTPUT A — BACKTEST COMPARISON

Menjawab:

Seberapa dekat hasil Python Simulator dengan MT5 Strategy Tester?

OUTPUT B — TRADING BEHAVIOR TEMPLATE

Menjawab:

Bagaimana kebiasaan market berdasarkan historical behavior dan kondisi tertentu, dan bagaimana pola tersebut dapat dijadikan referensi untuk trading berikutnya?

3. PRODUCT SCOPE

In Scope

- MQL5 source parser
- EA parameter extraction
- Variable extraction
- Function extraction
- Indicator detection
- `CopyBuffer()` mapping
- Trading function detection
- Strategy IR
- Indicator library
- Signal library
- Market library
- Trading library
- Python backtesting
- Visual simulator
- OHLC simulation
- M1-assisted execution
- Tick simulation
- Spread simulation
- Slippage simulation
- Commission
- Swap
- Margin
- Hedging
- Netting
- Pending orders

- SL/TP
- Trailing stop
- Grid/multiple position
- MT5 CSV import
- MT5/Python comparison
- Trade matching
- Equity comparison
- Discrepancy detection
- Behavior analysis
- Statistical market analysis
- AI behavior interpretation
- Behavior template storage
- Template matching
- CSV/JSON report

Out of Scope

- AI code explanation
- AI automatic MQL5 coding pada tahap awal
- AI yang menentukan profit guarantee
- Automatic live trading tanpa validation
- Broker execution replacement
- Guaranteed tick-level parity untuk data yang tidak tersedia

4. CORE PRINCIPLE

Deterministic Engine

Semua perhitungan backtest harus deterministic.

```

Historical Data
  ↓
Python Engine
  ↓
Deterministic Result

```

AI tidak boleh menentukan:

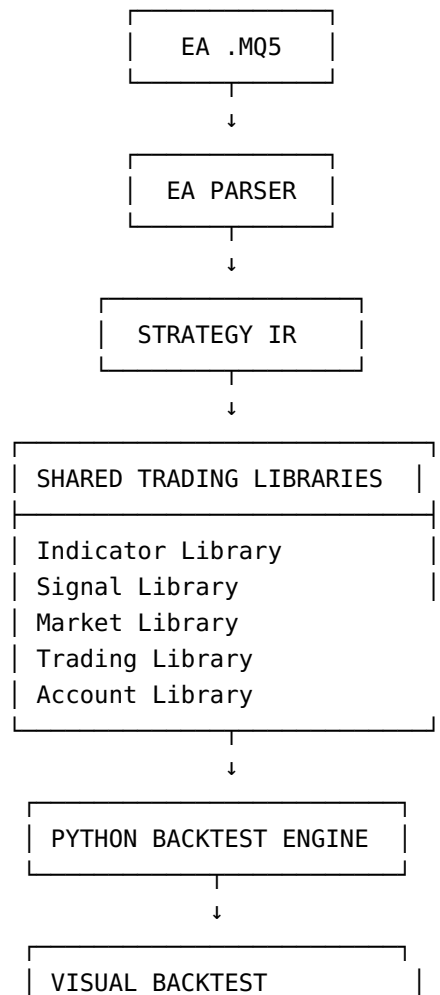
- entry price
- exit price
- profit
- balance
- equity
- commission

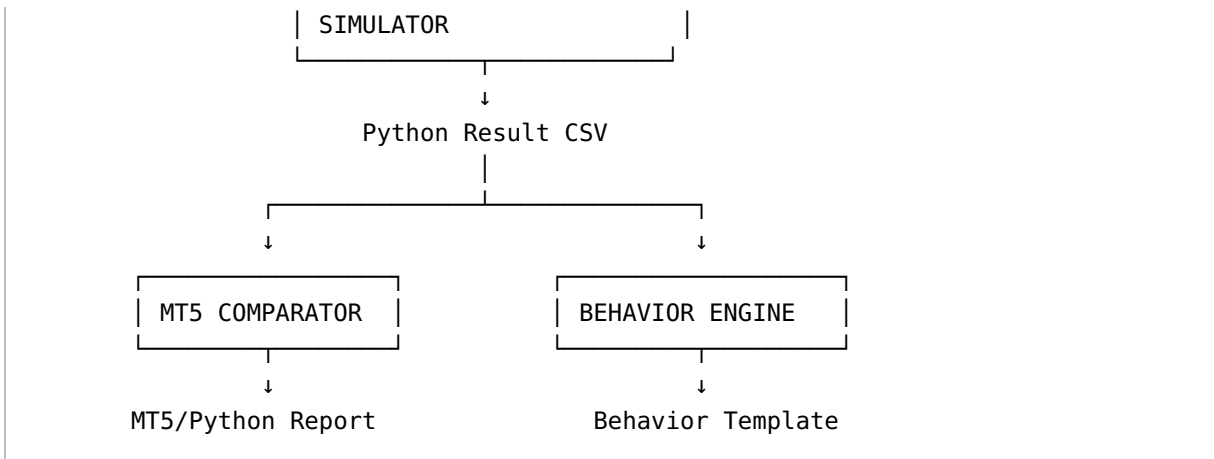
- spread
- indicator value

AI hanya digunakan untuk:

Historical Behavior
↓
Statistical Features
↓
AI Interpretation
↓
Behavior Template

5. SYSTEM ARCHITECTURE





6. MODULE 1 — EA PARSER

6.1 Purpose

Membaca source code MQL5 dan mengubahnya menjadi struktur Strategy IR.

6.2 Parser Responsibilities

Parser harus mengenali:

EA Metadata

- EA name
- version
- copyright
- description
- properties

Inputs

Contoh:

```
input double RiskPercent = 1.0;
input int ATRPeriod = 14;
input bool EnableTrailing = true;
```

Output:

```

{
  "RiskPercent": {
    "type": "double",
    "default": 1.0
  },
  "ATRPeriod": {
    "type": "int",
    "default": 14
  },
  "EnableTrailing": {
    "type": "bool",
    "default": true
  }
}

```

Variables

- global variables
- local variables
- static variables
- arrays
- handles
- enums
- structs
- constants

Functions

- OnInit
- OnTick
- OnTimer
- OnTrade
- OnTradeTransaction
- OnDeinit
- custom functions

Indicators

Minimum:

- iMA
- iRSI
- iATR
- iMACD
- iStochastic
- iADX
- iBands

- iCCI
- iMomentum
- iMFI
- iOBV
- iSAR
- iFractals
- iIchimoku

Buffer Access

Parser harus mengenali:

```
CopyBuffer()
CopyRates()
CopyClose()
CopyOpen()
CopyHigh()
CopyLow()
CopyTickVolume()
```

Trading

Parser harus mendeteksi:

```
CTrade
trade.Buy()
trade.Sell()
trade.PositionClose()
trade.PositionModify()
OrderSend()
OrderCheck()
```

Market Information

```
SymbolInfoDouble()
SymbolInfoInteger()
SymbolInfoTick()
```

7. MODULE 2 — COMMON STRATEGY IR

Parser tidak langsung menjalankan MQL5.

MQ5:

```
MQ5
↓
Lexer
↓
Parser
↓
AST
↓
Semantic Analyzer
↓
Strategy IR
```

Strategy IR menjadi bahasa internal platform.

Contoh:

```
{
  "strategy": {
    "name": "RSI ATR EA"
  },
  "indicators": [
    {
      "id": "rsi_1",
      "type": "RSI",
      "period": 14,
      "timeframe": "H1"
    },
    {
      "id": "atr_1",
      "type": "ATR",
      "period": 14,
      "timeframe": "H1"
    }
  ],
  "entry_rules": [],
  "exit_rules": [],
  "risk_management": {}
}
```

8. MODULE 3 — INDICATOR LIBRARY

Indicator Library menjadi shared dependency untuk:

```
EA Parser
EA Builder
Backtest Engine
Simulator
Behavior Engine
```

Trend

- SMA
- EMA
- WMA
- SMMA
- LWMA
- HMA
- DEMA
- TEMA

Momentum

- RSI
- MACD
- Stochastic
- CCI
- Williams %R
- ROC
- Momentum

Volatility

- ATR
- Bollinger Bands
- Standard Deviation
- Keltner Channel
- Donchian Channel

Trend Strength

- ADX
- +DI
- -DI

- Aroon

Volume

- OBV
- MFI
- Accumulation/Distribution
- VWAP

Price Action

- Highest
- Lowest
- Pivot
- Fractal
- Swing High
- Swing Low
- Candle Pattern

Setiap indicator harus mempunyai:

```
name
aliases
category
parameters
outputs
timeframe support
MQL5 mapping
calculation implementation
validation test
```

9. MODULE 4 — SIGNAL LIBRARY

Signal engine:

- Crossover
- Crossunder
- Greater Than
- Less Than
- Equal
- Above
- Below
- Rising
- Falling

- Breakout
- Breakdown
- Retest
- Reversal
- Candle pattern
- Composite condition

Contoh:

EMA21 crosses above EMA50

menjadi:

```
{
  "signal": "CROSSOVER",
  "fast": "EMA21",
  "slow": "EMA50"
}
```

10. MODULE 5 — MARKET LIBRARY

Market abstraction:

```
Symbol
Bid
Ask
Spread
Point
Digits
Contract Size
Tick Size
Tick Value
Volume
Time
Session
Timeframe
```

MTL5 compatibility:

```
SymbolInfoDouble(_Symbol, SYMBOL_BID)
```

dipetakan ke:

market.bid

11. MODULE 6 — TRADING LIBRARY

Menyediakan virtual broker:

Order
Deal
Position
Pending Order
SL
TP
Trailing
Magic Number

Mendukung:

- Market Buy
- Market Sell
- Buy Limit
- Sell Limit
- Buy Stop
- Sell Stop
- Stop Limit
- Position Modify
- Position Close
- Partial Close

12. MODULE 7 — ACCOUNT ENGINE

Mensimulasikan:

Balance
Equity
Margin
Free Margin
Margin Level

Floating Profit
Realized Profit
Commission
Swap
Stop Out

Account mode:

- Hedging
- Netting

13. MODULE 8 — PYTHON BACKTEST ENGINE

Backtest pipeline:

```
Load Data
↓
Initialize Account
↓
Initialize Strategy
↓
Initialize Indicators
↓
Process Market Data
↓
Evaluate Strategy
↓
Generate Order
↓
Execution Simulation
↓
Position Management
↓
SL/TP/Trailing
↓
Account Update
↓
Record Trade
↓
Next Tick/Candle
```

14. EXECUTION MODES

Mode 1 — OHLC

Fast testing menggunakan OHLC.

Mode 2 — M1 Execution

Strategy timeframe dapat berbeda dengan execution timeframe.

Contoh:

```
Strategy = H1  
Execution = M1
```

Mode 3 — Tick

Jika historical tick tersedia:

```
Bid  
Ask  
Timestamp  
Volume
```

Simulator memproses tick secara berurutan.

15. INTRABAR EXECUTION

Karena OHLC tidak selalu mengetahui urutan High/Low, engine harus memiliki execution policy.

Contoh:

```
OHLC Path A  
Open → High → Low → Close  
  
OHLC Path B  
Open → Low → High → Close
```

Mode:

- Conservative
- Optimistic
- OHLC heuristic
- M1 reconstruction
- Real tick

Setiap hasil harus mencatat execution mode yang digunakan.

16. VISUAL BACKTEST SIMULATOR

Simulator meniru MT5 Visual Tester.

Controls:

Play
Pause
Stop
Previous
Next Candle
Next Tick
Speed
Jump to Date
Jump to Trade

Speed:

0.25x
0.5x
1x
2x
5x
10x
50x
100x

17. SIMULATOR CHART

Menampilkan:

- Candlestick
- Volume
- Indicator
- BUY
- SELL
- Entry
- Exit
- SL
- TP
- Trailing Stop
- Pending Order
- Grid Position
- Current Price

Axis:

X-axis

Tanggal + waktu historical data.

Y-axis

Harga market.

18. SIMULATOR ACCOUNT PANEL

Initial Balance
Balance
Equity
Floating P/L
Margin
Free Margin
Margin Level
Drawdown

Semua berubah mengikuti simulation state.

19. SIMULATOR TRADE PANEL

Open Positions

Ticket
Symbol
Type
Volume
Entry
Current
SL
TP
Profit

Pending Orders

Ticket
Type
Price
Volume
SL
TP

History

Ticket
Open Time
Close Time
Type
Volume
Entry
Exit
Profit
Commission
Swap

20. MODULE 9 — MT5 COMPARATOR

Tujuan:

Menjadikan hasil MT5 Strategy Tester sebagai reference/ground truth.

Input:

MT5 Result CSV
Python Result CSV

21. CSV STANDARDIZATION

Kedua hasil dinormalisasi ke schema:

```
ticket
symbol
direction
volume
entry_time
entry_price
exit_time
exit_price
sl
tp
commission
swap
profit
```

22. TRADE MATCHING

Matching priority:

1. Symbol
2. Direction
3. Time proximity
4. Volume
5. Entry price
6. Exit time

Status:

```
MATCH
WARNING
MISMATCH
```

```
MISSING_MT5  
MISSING_PYTHON  
EXTRA_PYTHON
```

23. EQUITY COMPARISON

Per timestamp:

```
timestamp  
mt5_balance  
python_balance  
balance_difference  
mt5_equity  
python_equity  
equity_difference
```

Sistem harus mendeteksi:

First Divergence Point

Contoh:

```
First divergence:  
2026-01-05 13:00  
  
MT5 Equity:  
10,120  
  
Python:  
10,115  
  
Difference:  
5
```

24. DISCREPANCY ANALYSIS

Kategori:

- Entry time

- Entry price
- Exit time
- Exit price
- Volume
- SL
- TP
- Spread
- Commission
- Swap
- Indicator
- Intrabar execution
- Slippage
- Timezone
- Market data

25. MT5 PARITY SCORE

Metric:

Trade Matching
Entry Accuracy
Exit Accuracy
Volume Accuracy
SL Accuracy
TP Accuracy
Profit Accuracy
Equity Accuracy

Output:

Overall Parity Score

Score tidak boleh dianggap sebagai jaminan kesamaan absolut.

26. MODULE 10 — MARKET BEHAVIOR ENGINE

Behavior Engine menganalisis historical market.

Input:

OHLC
Tick
Spread
Volume
Indicators
Session
Economic Events
Trading History

Feature extraction:

Range
ATR
Volatility
Trend
Momentum
Drawdown
Maximum Excursion
Retracement
Breakout
False Breakout
Continuation
Reversal

27. EVENT BEHAVIOR ANALYSIS

Event contoh:

CPI
NFP
FOMC
PPI
Interest Rate
GDP
Unemployment
Powell Speech

Setiap event dianalisis berdasarkan window:

T-60m
T-30m

T-15m
T-5m
T0
T+5m
T+15m
T+30m
T+60m
T+120m

28. BEHAVIOR METRICS

Untuk setiap event:

Sample Size
Average Range
Median Range
Maximum Range
Minimum Range
Initial Direction
Retracement Probability
Continuation Probability
Reversal Probability
False Breakout Probability
Maximum Favorable Excursion
Maximum Adverse Excursion
Volatility Expansion

29. SESSION BEHAVIOR

Analisis:

Asian Session
London Session
New York Session
London/New York Overlap

Metric:

```
Average Range  
Trend Probability  
Breakout Probability  
Reversal Probability  
Volatility  
Average Move
```

30. MARKET REGIME

Engine mendeteksi:

```
Trending  
Range  
Compression  
Expansion  
High Volatility  
Low Volatility  
Breakout  
Pullback  
Reversal
```

Regime dapat dikombinasikan dengan event.

Contoh:

```
CPI + Compression  
CPI + Uptrend  
CPI + High Volatility  
CPI + Range
```

31. AI BEHAVIOR ANALYST

AI hanya menerima statistical feature dataset.

Contoh:

```
{  
  "event": "CPI",
```



```
"sample_size": 32,  
"initial_expansion_probability": 0.87,  
"retracement_probability": 0.68,  
"continuation_probability": 0.72  
}
```

AI menghasilkan:

```
Behavior Interpretation  
Market Context  
Observed Pattern  
Potential Trading Implication  
Risk Consideration
```

AI tidak boleh mengubah angka statistik.

32. TRADING BEHAVIOR TEMPLATE

Template:

```
{  
  "template_id": "XAUUSD_CPI_V1",  
  "symbol": "XAUUSD",  
  "event": "CPI",  
  "sample_size": 32,  
  
  "pre_event": {},  
  
  "reaction": {},  
  
  "retracement": {},  
  
  "continuation": {},  
  
  "direction": {},  
  
  "risk": {},  
  
  "confidence": 0.84,  
  
  "created_at": "...",  
}
```

```
"version": 1
}
```

33. TEMPLATE VERSIONING

Template tidak boleh ditimpa begitu saja.

Contoh:

```
XAUUSD_CPI_V1
XAUUSD_CPI_V2
XAUUSD_CPI_V3
```

Setiap update harus menyimpan:

```
Sample Size
Previous Version
New Evidence
Changed Metrics
Confidence Change
Creation Date
```

34. TEMPLATE VALIDATION

Template baru harus diuji menggunakan out-of-sample data.

```
Historical Data
  ↓
Training Period
  ↓
Behavior Template
  ↓
Validation Period
  ↓
Template Validation
```

Status:

DRAFT
VALIDATED
STABLE
DEPRECATED

35. TEMPLATE USAGE

Ketika kondisi market baru muncul:

```
Current Market
  ↓
Feature Extraction
  ↓
Regime Detection
  ↓
Event Detection
  ↓
Template Matching
  ↓
Historical Behavior Reference
```

Output:

```
Matching Template:
XAUUSD_CPI_V2

Similarity:
87%

Historical Sample:
41 events

Observed Behavior:
Initial expansion → retracement → continuation
```

Template merupakan **reference**, bukan jaminan arah market.

36. DUA OUTPUT UTAMA PLATFORM

OUTPUT 1 — BACKTEST COMPARISON

```
Python Simulator
+
MT5 Strategy Tester
↓
Trade Comparison
Equity Comparison
Statistics Comparison
Discrepancy Detection
↓
comparison.csv
summary.csv
equity_comparison.csv
discrepancies.csv
```

OUTPUT 2 — TRADING BEHAVIOR TEMPLATE

```
Historical Market
↓
Statistical Analysis
↓
Behavior Detection
↓
AI Interpretation
↓
Trading Behavior Template
↓
Behavior Database
```

37. FILE STRUCTURE

```
ea-backtest-engine-v3/
├─ app.py
├─
├─ parser/
│   └─ lexer.py
```

```
|
|   ├── parser.py
|   ├── ast.py
|   ├── semantic_analyzer.py
|   ├── variable_extractor.py
|   ├── function_extractor.py
|   ├── indicator_extractor.py
|   ├── trading_extractor.py
|   └── strategy_ir.py
|
|── libraries/
|   ├── indicators/
|   ├── signals/
|   ├── market/
|   ├── trading/
|   └── account/
|
|── runtime/
|   ├── runtime.py
|   ├── context.py
|   └── mt5_compatibility.py
|
|── backtest/
|   ├── engine.py
|   ├── data_feed.py
|   ├── market_engine.py
|   ├── execution_engine.py
|   ├── order_engine.py
|   ├── position_engine.py
|   ├── account_engine.py
|   └── statistics.py
|
|── simulator/
|   ├── session.py
|   ├── playback.py
|   ├── candle_player.py
|   ├── tick_player.py
|   └── state.py
|
|── comparator/
|   ├── mt5_loader.py
|   ├── python_loader.py
|   ├── normalizer.py
|   ├── trade_matcher.py
|   ├── equity_comparator.py
|   ├── discrepancy.py
|   └── parity.py
|
|── behavior/
```

```
| | feature_engine.py
| | event_analyzer.py
| | regime_detector.py
| | pattern_detector.py
| | behavior_engine.py
| | ai_behavior.py
| | template_engine.py
| | └─ template_validator.py
|
| └─ reports/
|   | backtest_report.py
|   | comparison_report.py
|   | └─ behavior_report.py
|
| └─ api/
|   | parser_routes.py
|   | backtest_routes.py
|   | simulator_routes.py
|   | comparison_routes.py
|   | └─ behavior_routes.py
|
| └─ frontend/
|   | └─ index.html
|
| └─ data/
|   | market/
|   | mt5_results/
|   | └─ behavior_templates/
|
| └─ tests/
```

38. API REQUIREMENTS

Parse EA

```
POST /api/parse-ea
```

Input:

```
{
  "filename": "my_ea.mq5",
```

```
  "source": "..."  
}
```

Output:

```
{  
  "success": true,  
  "strategy_ir": {},  
  "inputs": {},  
  "indicators": [],  
  "functions": [],  
  "warnings": []  
}
```

Run Backtest

```
POST /api/run-backtest
```

Input:

```
{  
  "strategy_id": "...",  
  "symbol": "XAUUSD",  
  "timeframe": "H1",  
  "data": "...",  
  "initial_balance": 10000,  
  "execution_mode": "M1"  
}
```

Start Simulator

```
POST /api/simulator/start
```

Simulator Tick

```
POST /api/simulator/next
```

MT5 Comparison

```
POST /api/compare-mt5
```

Input:

```
{
  "mt5_result": "...",
  "python_result": "..."
}
```

Analyze Behavior

```
POST /api/analyze-behavior
```

Input:

```
{
  "symbol": "XAUUSD",
  "event": "CPI",
  "history": "..."
}
```

Match Template

```
POST /api/behavior/match
```

39. DATABASE

Recommended entities:

```
EA
EAInput
EAIndicator
EAFunction
StrategyIR
BacktestSession
Trade
Order
Position
AccountSnapshot
MT5Result
ComparisonResult
BehaviorSample
BehaviorFeature
BehaviorTemplate
BehaviorTemplateVersion
```

40. BACKTEST SESSION

Every backtest must have:

```
session_id
EA
symbol
timeframe
data_source
start_date
end_date
initial_balance
execution_mode
spread_mode
slippage_mode
commission
swap
created_at
```

This makes every result reproducible.

41. REPRODUCIBILITY

Setiap backtest harus menyimpan:

```
EA source hash
Strategy IR hash
Historical data hash
Configuration hash
Engine version
Indicator library version
Execution mode
Random seed
```

Tujuan:

Backtest yang sama harus menghasilkan hasil yang sama.

42. TESTING STRATEGY

Unit Tests

- Indicator
- Signal
- Parser
- Order
- Position
- Account
- Comparator

Integration Tests

```
EA
→ Parser
→ Strategy IR
→ Simulator
→ Result
```

MT5 Parity Tests

MT5
vs
Python

Target awal:

Trade matching > 95%

Target mature:

Trade matching > 99%

Tergantung kualitas historical data dan execution model.

43. ACCEPTANCE CRITERIA

EA Parser

Parser dianggap berhasil apabila dapat:

- membaca EA `.mq5`
- mengekstrak seluruh `input`
- mengenali variable
- mengenali function
- mengenali indicator
- mengenali `CopyBuffer`
- mengenali trading function
- menghasilkan Strategy IR
- memberikan warning terhadap construct yang belum didukung

Parser **tidak boleh mengklaim 100% kompatibilitas** apabila terdapat MQL5 construct yang belum dapat dipetakan.

Simulator

Simulator dianggap berhasil apabila:

- candle/tick dapat diputar

- pause/resume bekerja
 - order dapat dibuat
 - position dapat dibuat
 - SL/TP bekerja
 - trailing bekerja
 - balance berubah
 - equity berubah
 - trade history tercatat
 - chart menampilkan trade marker
-

Comparator

Harus menghasilkan:

```
trade_comparison.csv  
equity_comparison.csv  
summary.csv  
discrepancies.csv
```

dan menemukan:

First divergence point.

Behavior Engine

Harus dapat:

- mengelompokkan historical market condition
 - menghitung statistik
 - menganalisis event
 - menganalisis session
 - mendeteksi regime
 - membuat behavior template
 - versioning template
 - melakukan validation
 - melakukan template matching
-

44. DEVELOPMENT PHASE

Phase 1 — Foundation

- Repository restructuring
- Core data models
- Strategy IR
- CSV schema
- Database

Phase 2 — EA Parser

- Lexer
- Parser
- Input extraction
- Variable extraction
- Function extraction
- Indicator extraction
- Trading extraction

Phase 3 — Indicator Library

- EMA
- SMA
- RSI
- ATR
- MACD
- Bollinger
- Stochastic
- ADX
- Fractal
- Highest/Lowest
- Crossover/Crossunder

Kemudian diperluas.

Phase 4 — Python Backtest Engine

- Market engine
- Order engine
- Position engine
- Account engine
- SL/TP
- Spread
- Commission
- Swap

- Slippage

Phase 5 — Visual Simulator

- Playback
- Chart
- Orders
- Positions
- Account
- Trade history

Phase 6 — MT5 Comparator

- MT5 CSV import
- Normalization
- Trade matcher
- Equity comparison
- Discrepancy detection
- Parity score

Phase 7 — Behavior Engine

- Feature extraction
- Event analysis
- Session analysis
- Regime detection
- Pattern detection

Phase 8 — AI Behavior Analyst

- Statistical feature input
- Behavior interpretation
- Template generation
- Template validation

Phase 9 — Integration

```
EA
→ Parser
→ Strategy IR
→ Simulator
→ MT5 Comparison
→ Behavior Analysis
→ Template
```

45. SECURITY & DATA INTEGRITY

Source EA dan historical trading data harus diperlakukan sebagai user data.

Sistem tidak boleh:

- mengekspos API key
- mengeksekusi arbitrary system command dari EA source
- menjalankan MQL5 executable secara langsung tanpa sandbox
- memberikan akses filesystem bebas kepada EA

EA Runtime harus menggunakan **sandboxed virtual environment**.

46. IMPORTANT DESIGN DECISION

Platform memiliki tiga lapisan berbeda:

LAYER 1
STRATEGY
EA Parser
EA Builder
Strategy IR

LAYER 2
EXECUTION
Market
Indicators
Orders
Positions
Account
Simulator

LAYER 3
INTELLIGENCE
MT5 Comparison
Behavior Analysis
Behavior Template

AI hanya berada pada:

LAYER 3
Behavior Analysis

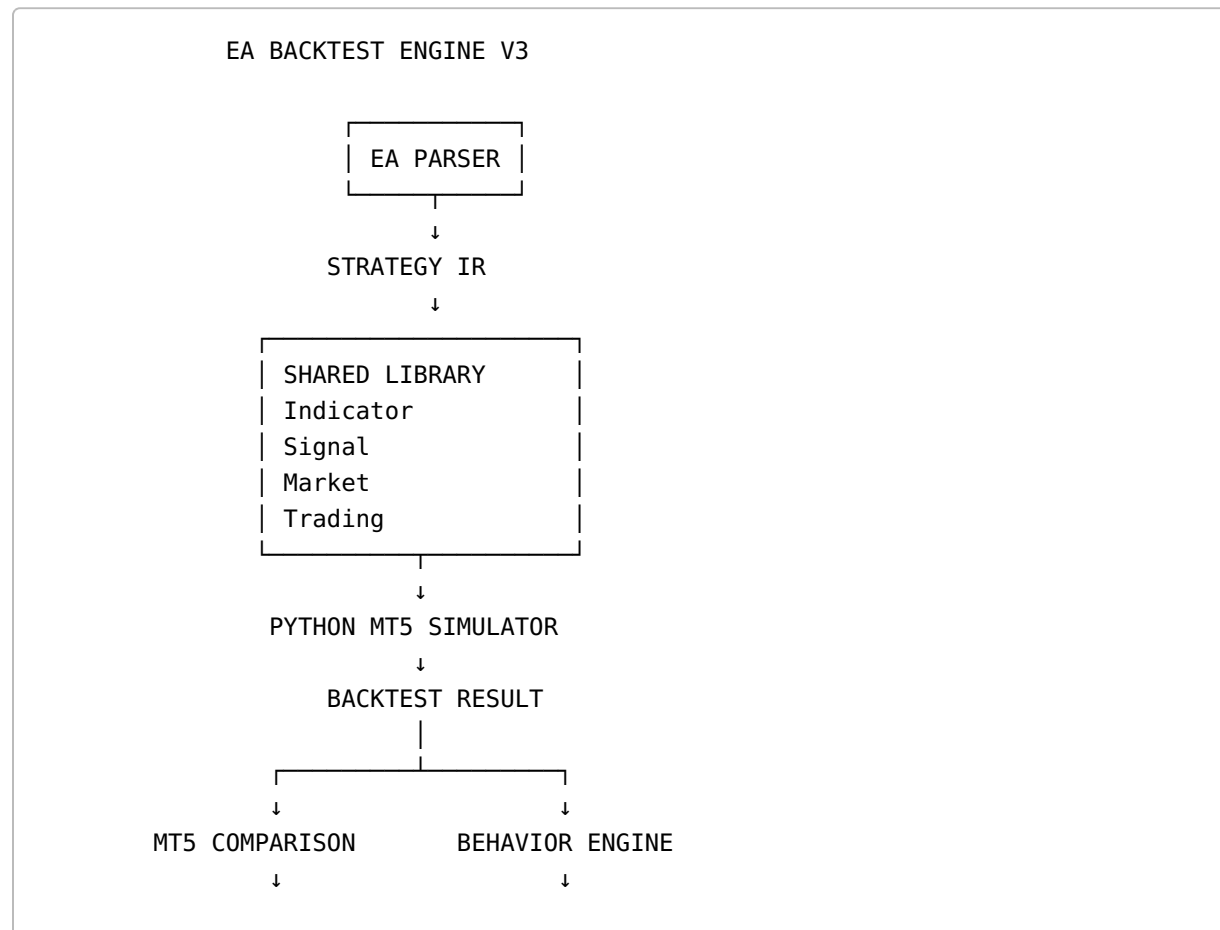
Bukan pada:

EA Parser
Backtest Calculation
Order Execution
Statistics
MT5 Comparison

47. FINAL PRODUCT DEFINITION

EA Backtest Engine V3 bukan sekadar Python backtesting library.

Produk ini terdiri dari:



OUTPUT #1
CSV REPORT

OUTPUT #2
BEHAVIOR TEMPLATE

OUTPUT #1

Trading Comparison

MT5 vs Python
Trade
Equity
Profit
Execution
Statistics
Discrepancy
Parity Score

OUTPUT #2

Trading Behavior Template

Historical Market Behavior
Event Behavior
Session Behavior
Trend Behavior
Volatility Behavior
Price Action Behavior
Regime
Probability
Historical Range
Retracement
Continuation
Reversal
Risk Characteristics

Template tersebut kemudian menjadi **historical reference layer** untuk analisis kondisi market berikutnya.

48. SUCCESS METRIC

Produk dianggap berhasil apabila:

1. EA dapat diparse menjadi Strategy IR.
2. Indicator yang digunakan EA dapat dikenali dan dihitung oleh shared library.

3. EA dapat dijalankan dalam Python simulator.
4. Simulator dapat diputar secara visual.
5. Hasil dapat diekspor ke CSV.
6. Hasil MT5 dapat diimpor.
7. Python vs MT5 dapat dibandingkan trade-by-trade.
8. Sistem dapat menemukan first divergence.
9. Historical market behavior dapat dianalisis.
10. Behavior template dapat dibuat dan disimpan.
11. Template dapat dicocokkan dengan kondisi market berikutnya.
12. Semua hasil backtest tetap deterministic.
13. AI hanya digunakan untuk interpretasi behavior, bukan untuk mengubah hasil perhitungan trading.

49. TARGET AKHIR

Target akhir sistem adalah:

"Build a Python-based MT5-compatible Strategy Tester that can parse MQL5 Expert Advisors, execute their strategy through a shared indicator/trading library, visually replay historical markets, validate its results against real MT5 Strategy Tester results, and transform historical market behavior into reusable Trading Behavior Templates for future analysis and trading reference."

Dengan demikian sistem memiliki dua mesin utama:

ENGINE A

MT5-Compatible Backtest
+ Visual Simulator
+ MT5 Comparator

ENGINE B

Market Behavior Analysis
+ AI Interpretation
+ Behavior Template
+ Historical Reference

Keduanya menggunakan historical data yang sama tetapi memiliki tujuan berbeda: **Engine A menguji apakah strategi bekerja dan apakah simulator mereplikasi MT5; Engine B mempelajari bagaimana market biasanya berperilaku dan menyimpan pola tersebut sebagai referensi.**